Edge Detection

Derivatives and Edges

An edge in an image $I(x, y)$ corresponds to a location where the intensity function changes rapidly. We can detect this using derivatives:

First-order derivative: Measures the rate of change

$$\frac{\partial I}{\partial x}, \quad \frac{\partial I}{\partial y}$$

Second-order derivative: Measures the rate of change of the rate of change

$$\frac{\partial^2 I}{\partial x^2}, \quad \frac{\partial^2 I}{\partial y^2}$$

Since images are discrete, we approximate these derivatives using finite differences and convolution kernels.

Discrete Approximation of Derivatives

From Continuous to Discrete

For a continuous function, the derivative is:

$$\frac{df}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

For discrete images, we use finite differences:

$$\frac{\partial I}{\partial x} \approx I(x+1, y) - I(x, y)$$

or:

$$\frac{\partial I}{\partial x} \approx \frac{I(x+1, y) - I(x-1, y)}{2}$$

This motivates the design of gradient kernels.

Roberts Cross Operator

Roberts approximates gradients along diagonal directions:

- Computes differences along 45° and 135° diagonals
- Smallest possible kernel (2×2)
- Approximates: $I(x+1, y+1) - I(x, y)$

```
roberts_x = np.array([[1, 0],[0, -1]])
roberts_y = np.array([[0, 1],[-1, 0]])
rx = cv2.filter2D(img, -1, roberts_x)
ry = cv2.filter2D(img, -1, roberts_y)
roberts = cv2.magnitude(rx.astype(float),
ry.astype(float))
```

Kernels:

$$G_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \quad G_y = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

- G_x: Diagonal difference (NW-SE)
- G_y: Diagonal difference (NE-SW)

Roberts Edge Detection

Prewitt Operator

Prewitt extends the finite difference by averaging over three rows/columns:

- Central difference: $I(x+1,y) - I(x-1,y)$
- Averaged over neighboring rows
- Each pixel weighted equally
- Provides smoothing perpendicular to gradient direction

```
kernel_x = np.array([[1, 0, -1],[1, 0, -1],[1, 0,
-1]])
kernel_y = np.array([[1, 1, 1],[0, 0, 0],[-1, -1,
-1]])
prewitt_x = cv2.filter2D(img, -1, kernel_x)
prewitt_y = cv2.filter2D(img, -1, kernel_y)
prewitt = cv2.magnitude(prewitt_x.astype(float),
prewitt_y.astype(float))
```

Kernels:

$$G_x = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \qquad G_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Horizontal gradient averaged over 3 rows; vertical gradient averaged over 3 columns.

Prewitt Edge Detection

Sobel Operator

Sobel improves on Prewitt by using Gaussian-like weighting:

- Central difference with weighted averaging
- Center row/column weighted by 2
- Approximates Gaussian smoothing
- Derived from separable kernels: $[1\ 2\ 1]^T \times [1\ 0\ -1]$

Kernels:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

The weights [1, 2, 1] approximate a Gaussian distribution.

```
sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)

sobel = cv2.magnitude(sobel_x, sobel_y)
```

Sobel Edge Detection

Gradient Magnitude and Direction

For any gradient-based operator, we compute:

Gradient magnitude (edge strength):

$$G = \sqrt{G_x^2 + G_y^2}$$

Edge direction (perpendicular to gradient):

$$\theta = \tan^{-1}\left(\frac{G_y}{G_x}\right)$$

This represents the direction and magnitude of the maximum rate of change in the image at each point.

Laplacian Operator

First derivatives detect slopes (gradients). Second derivatives detect where slopes change (inflection points).

The Laplacian Operator

The Laplacian is the sum of second partial derivatives:

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

Properties:

- Isotropic (rotationally invariant)
- Detects zero-crossings (where the second derivative changes sign)
- Zero-crossings correspond to edges

Discrete Laplacian Kernel

The second derivative in 1D:

$$\frac{\partial^2 I}{\partial x^2} \approx I(x+1) - 2I(x) + I(x-1)$$

Similarly for y. The 2D Laplacian becomes:

$$\nabla^2 I \approx [I(x+1,y) + I(x-1,y) + I(x,y+1) + I(x,y-1)] - 4I(x,y)$$

This yields the kernel:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Each neighbor weighted -1, center weighted +4 (sum = 0).

Laplacian Operator Implementation

The Discrete Laplacian

$$\nabla^2 I = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Alternative with diagonal neighbors:

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

```
laplacian = cv2.Laplacian(img, cv2.CV_64F)
```

Direct Implementation

Edge detection via zero-crossings or thresholding the Laplacian response.

Laplacian Edge Detection

Laplacian of Gaussian (LoG)

Combining Smoothing and Edge Detection

The LoG applies Gaussian smoothing before the Laplacian:

$$\text{LoG} = \nabla^2[G(x,y) * I(x,y)]$$

where $G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$

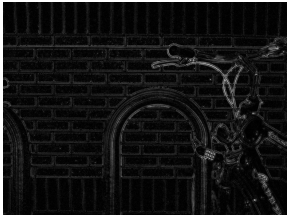
```
blur = cv2.GaussianBlur(img, (5, 5), 0)
log = cv2.Laplacian(blur, cv2.CV_64F)
```

Mathematical Justification

- Gaussian removes high-frequency noise
- Laplacian then detects edges via second derivative
- By linearity of convolution: $\nabla^2(G * I) = (\nabla^2 G) * I$
- Can precompute $\nabla^2 G$ as a single kernel (LoG kernel)

Edges are found at zero-crossings of the LoG response.

Laplacian of Gaussian



Laplacian of Gaussian Implementation

Two-step process:

1. Apply Gaussian blur
2. Apply Laplacian

Equivalently, convolve with the LoG kernel directly.

The standard deviation σ of the Gaussian controls the scale of edges detected.

Canny Edge Detector: Multi-Stage Approach

A Principled Algorithm

Canny designed an optimal edge detector based on three criteria:

1. Good detection (minimize false positives and negatives)
2. Good localization (edges close to true edges)
3. Single response (one detection per edge)

Algorithm Steps

1. **Gaussian smoothing**: Remove noise
2. **Gradient computation**: Use Sobel to find magnitude and direction
3. **Non-maximum suppression**: Thin edges to single-pixel width
4. **Double thresholding**: Classify edges as strong, weak, or non-edges
5. **Edge tracking by hysteresis**: Connect weak edges to strong edges

Canny: Non-Maximum Suppression

Logic: Thinning Edges to Single Pixels

After computing gradient magnitude and direction:

- For each pixel, check if it's a local maximum along the gradient direction
- Compare with neighbors in the direction of the gradient
- Suppress (set to zero) if not a local maximum
- Result: edges are thinned to single-pixel width

This addresses the "single response per edge" criterion by ensuring each edge produces only one detection along its normal direction.

Canny: Double Thresholding and Hysteresis

Logic: Robust Edge Linking
Double thresholding:

- High threshold T_h: Pixels above this are "strong edges"
- Low threshold T_l: Pixels above this but below T_h are "weak edges"
- Pixels below T_l are discarded

Edge tracking by hysteresis:

- Start with strong edge pixels (certain edges)
- Recursively follow weak edge pixels connected to strong edges
- Weak edges not connected to strong edges are discarded

This provides better connectivity and reduces false edges compared to single thresholding.

Canny Implementation and Results

Complete Pipeline:

1. Gaussian blur (σ)
2. Sobel gradients
3. Non-max suppression
4. Hysteresis thresholding (T_l, T_h)

```
canny = cv2.Canny(img, threshold1=100,
                  threshold2=200)
```

Parameters σ, T_l, T_h control sensitivity and edge continuity.

Canny produces thin, well-localized, and well-connected edges.

Canny Edge Detection